

DEBER 8 - MEJORA ANALITICA

Analisis y validacion de registros de mantenimiento

Herramientas: Python, pandas, NumPy y Matplotlib

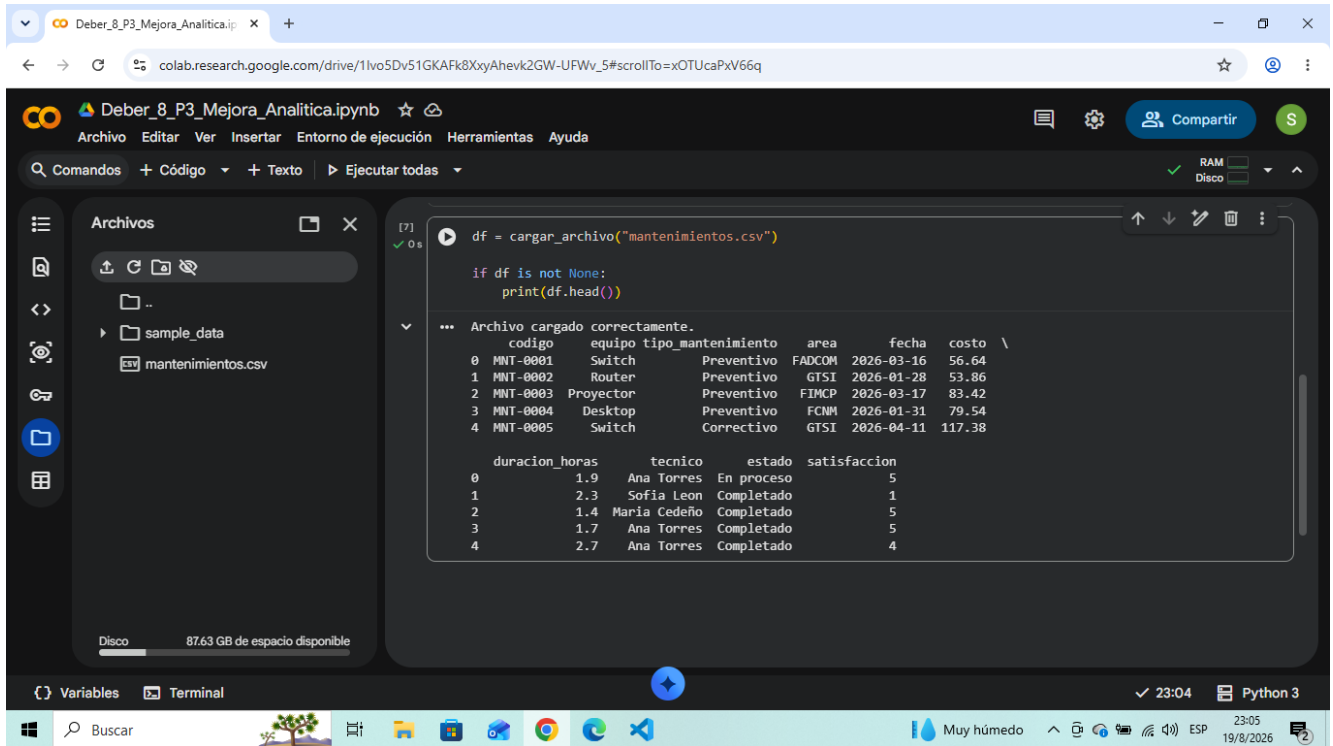
Entorno de trabajo: Google Colab / Jupyter Notebook

Documento de evidencias

Este documento presenta las capturas y una explicacion breve del desarrollo realizado para cumplir los requisitos de la asignacion.

1. Lectura y manejo de archivos

Se creo la funcion **cargar_archivo(ruta)** utilizando pandas. La lectura se realiza dentro de un bloque **try/except** para controlar archivo inexistente, archivo vacio y errores de interpretacion del CSV. La prueba con **mantenimientos.csv** confirma que el archivo valido se carga correctamente.



```
[7] df = cargar_archivo("mantenimientos.csv")
[7] ✓ 0 s

if df is not None:
    print(df.head())

... Archivo cargado correctamente.
   codigo  equipo  tipo_mantenimiento  area  fecha  costo \
0  MNT-0001  Switch  Preventivo  FADCOM  2026-03-16  56.64
1  MNT-0002  Router  Preventivo  GTSI  2026-01-28  53.86
2  MNT-0003  Proyector  Preventivo  FIMCP  2026-03-17  83.42
3  MNT-0004  Desktop  Preventivo  FCNM  2026-01-31  79.54
4  MNT-0005  Switch  Correctivo  GTSI  2026-04-11  117.38

   duracion_horas  tecnico  estado  satisfaccion
0              1.9  Ana Torres  En proceso         5
1              2.3  Sofia Leon  Completado         1
2              1.4  Maria Cedeño  Completado         5
3              1.7  Ana Torres  Completado         5
4              2.7  Ana Torres  Completado         4
```

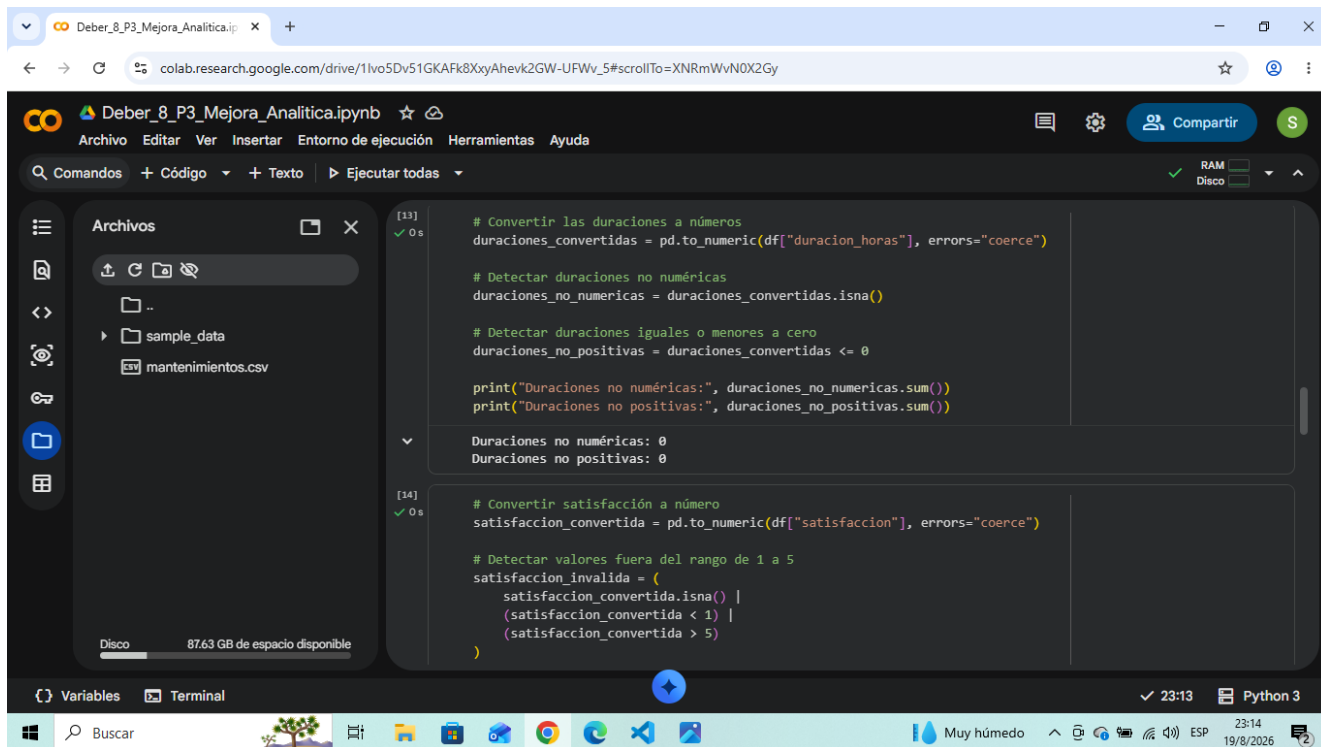
Disco 87.63 GB de espacio disponible

Variables Terminal

Muy húmedo 23:05 19/8/2026

2. Validacion de datos

Se validaron codigos vacios y duplicados, fechas invalidas, costos no numericos o negativos, duraciones no numericas o no positivas, satisfaccion fuera del rango de 1 a 5, estados no permitidos y categorias de equipo no contempladas. Con el archivo valido las reglas mostradas no detectan inconsistencias.



The screenshot shows a Google Colab notebook titled 'Deber_8_P3_Mejora_Analitica.ipynb'. The left sidebar shows a file explorer with 'sample_data' and 'mantenimientos.csv'. The main area displays two code cells. Cell [13] validates 'duracion_horas' by converting it to numeric, checking for non-numeric values, and detecting zero or negative durations. Cell [14] validates 'satisfaccion' by converting it to numeric and checking for values outside the 1 to 5 range. Both cells show successful execution with 0 seconds runtime.

```
[13] ✓ 0 s
# Convertir las duraciones a números
duraciones_convertidas = pd.to_numeric(df["duracion_horas"], errors="coerce")

# Detectar duraciones no numéricas
duraciones_no_numericas = duraciones_convertidas.isna()

# Detectar duraciones iguales o menores a cero
duraciones_no_positivas = duraciones_convertidas <= 0

print("Duraciones no numéricas:", duraciones_no_numericas.sum())
print("Duraciones no positivas:", duraciones_no_positivas.sum())

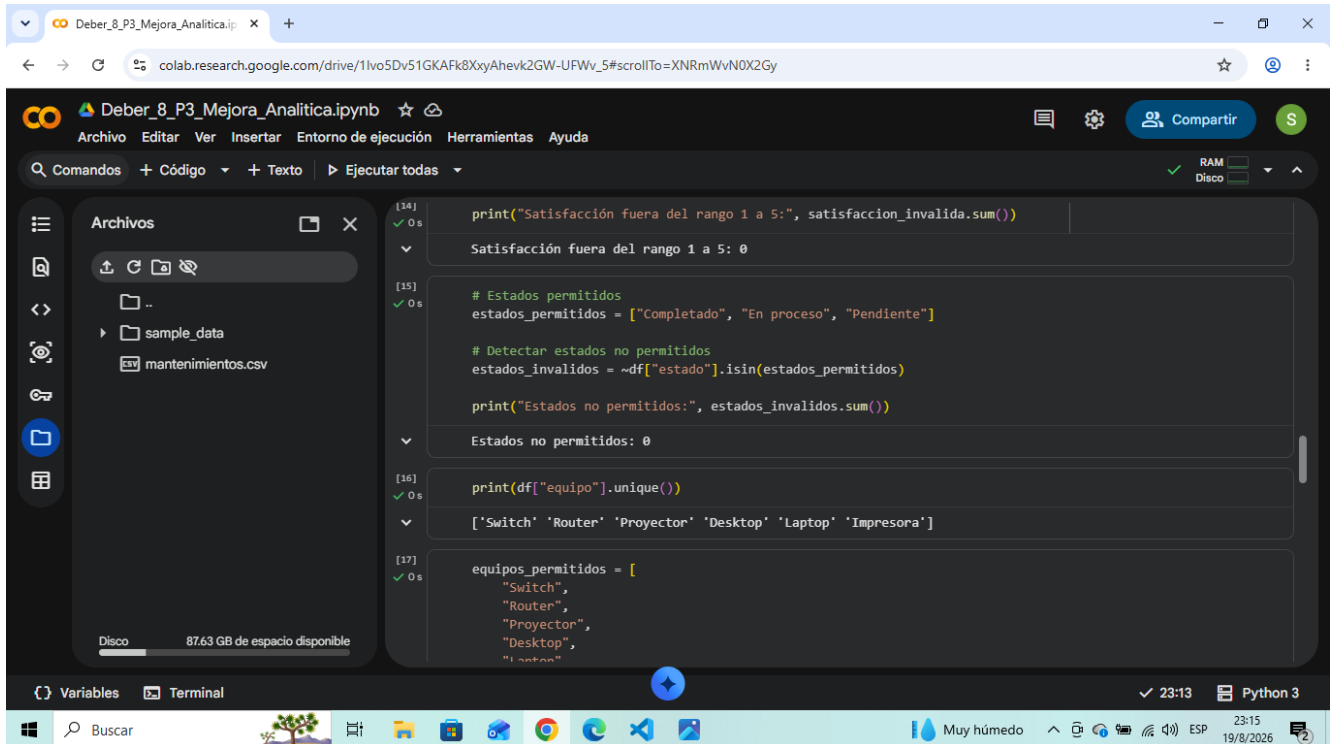
Duraciones no numéricas: 0
Duraciones no positivas: 0

[14] ✓ 0 s
# Convertir satisfacción a número
satisfaccion_convertida = pd.to_numeric(df["satisfaccion"], errors="coerce")

# Detectar valores fuera del rango de 1 a 5
satisfaccion_invalida = (
    satisfaccion_convertida.isna() |
    (satisfaccion_convertida < 1) |
    (satisfaccion_convertida > 5)
)
```

3. Estados y categorías permitidas

Para los estados se definieron como valores permitidos **Completado**, **En proceso** y **Pendiente**. Para los equipos se tomaron las categorías presentes en el conjunto valido: Switch, Router, Proyector, Desktop, Laptop e Impresora.



The screenshot shows a Google Colab notebook titled "Deber_8_P3_Mejora_Analitica.ipynb". The notebook is open in a web browser at the URL `colab.research.google.com/drive/1lvo5Dv51GKAFk8XyAhevk2GW-UPWv_5#scrollTo=XNRmWvN0X2Gy`. The notebook interface includes a file explorer on the left showing a folder named "sample_data" and a file named "mantenimientos.csv". The main area displays the following Python code:

```
[134] print("Satisfacción fuera del rango 1 a 5:", satisfaccion_invalida.sum())
      Satisfacción fuera del rango 1 a 5: 0

[135] # Estados permitidos
      estados_permitidos = ["Completado", "En proceso", "Pendiente"]

      # Detectar estados no permitidos
      estados_invalidos = ~df["estado"].isin(estados_permitidos)

      print("Estados no permitidos:", estados_invalidos.sum())

      Estados no permitidos: 0

[136] print(df["equipo"].unique())

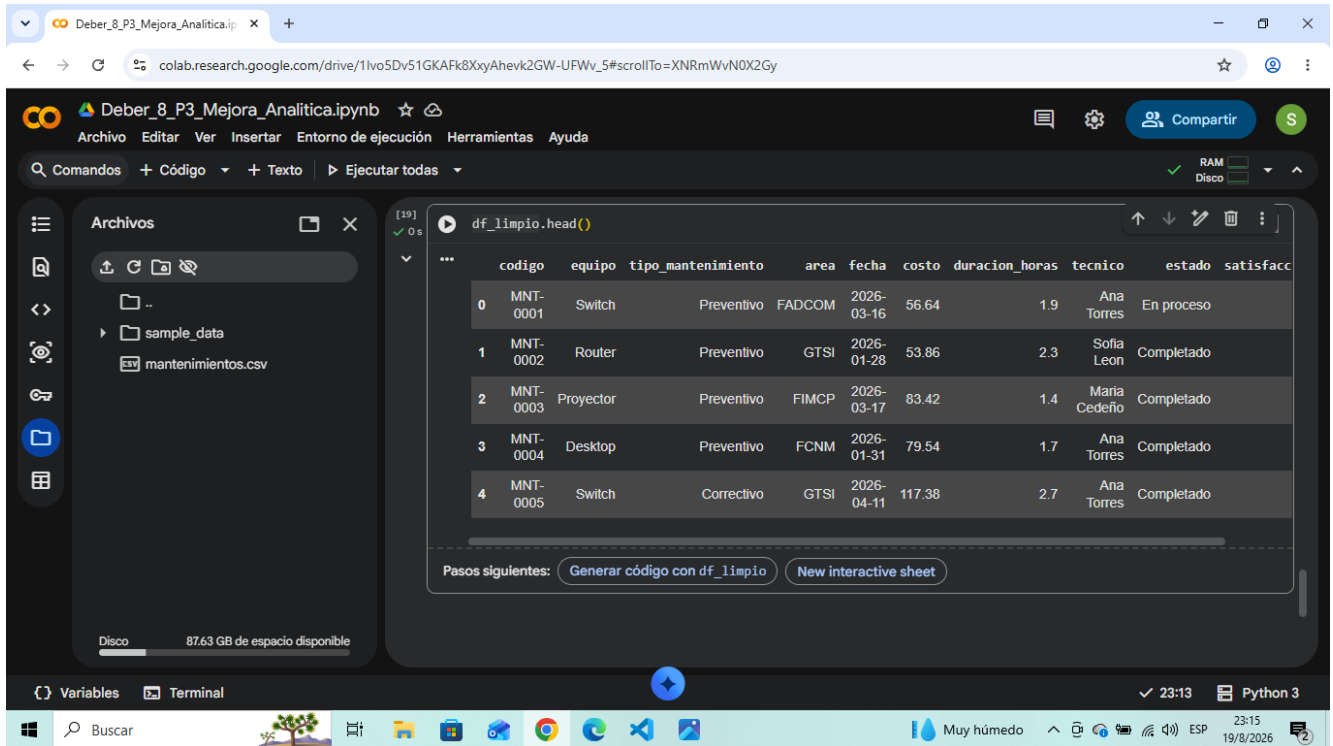
      ['Switch' 'Router' 'Proyector' 'Desktop' 'Laptop' 'Impresora']

[137] equipos_permitidos = [
      "Switch",
      "Router",
      "Proyector",
      "Desktop",
      "Laptop",
      "Impresora"]
```

The bottom of the notebook shows a status bar with "Variables", "Terminal", and "Python 3". The system tray at the bottom indicates the time is 23:15 on 19/8/2026, and the weather is "Muy húmedo".

4. DataFrame limpio

Las condiciones de error se combinaron para excluir los registros invalidos. El resultado se almaceno en **df_limpio**. La captura muestra las primeras filas del DataFrame limpio despues de aplicar las reglas de validacion.

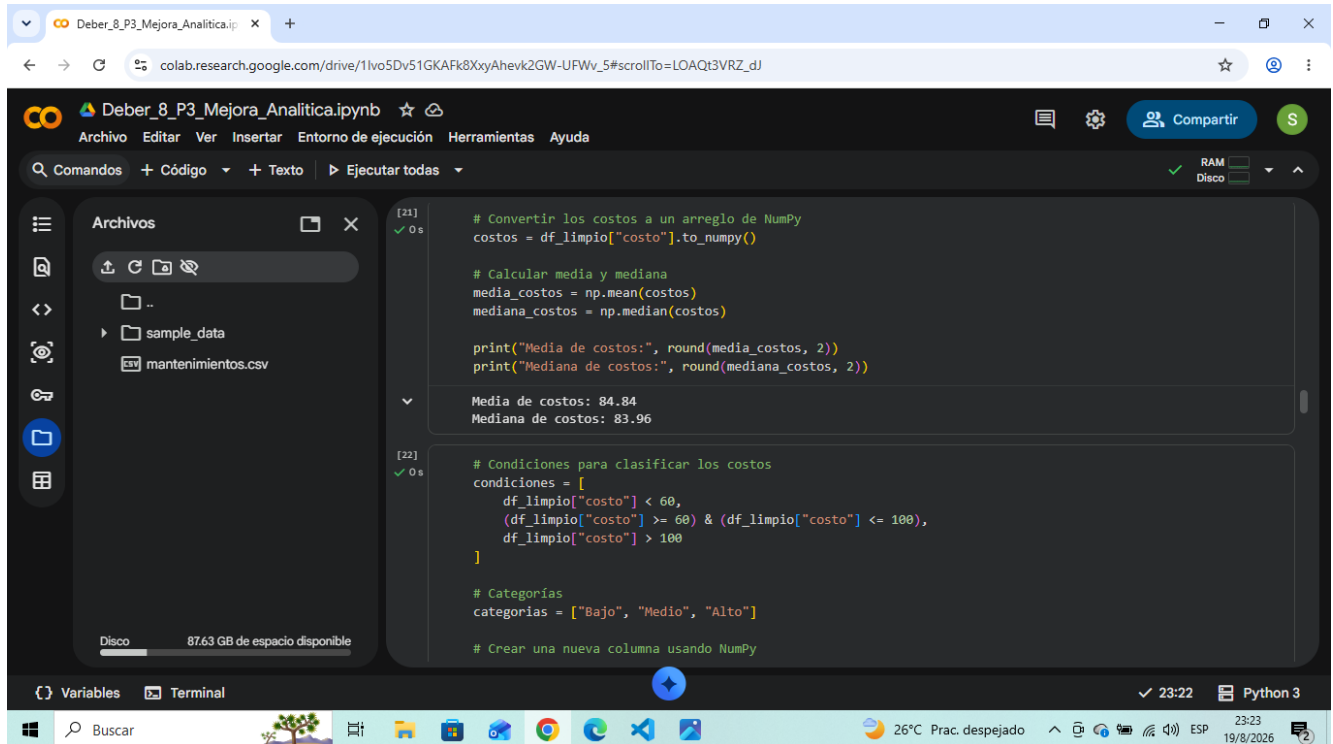


The screenshot shows a Google Colab notebook titled "Deber_8_P3_Mejora_Analitica.ipynb". The left sidebar displays the file explorer with a folder named "sample_data" containing a file "mantenimientos.csv". The main area shows the output of the command `df_limpio.head()`, which displays the first five rows of a cleaned DataFrame. The DataFrame has columns: `codigo`, `equipo`, `tipo_mantenimiento`, `area`, `fecha`, `costo`, `duracion_horas`, `tecnico`, `estado`, and `satisfacc`. Below the table, there are buttons for "Generar código con df_limpio" and "New interactive sheet". The bottom status bar indicates "Muy húmedo" and the date "19/8/2026".

	codigo	equipo	tipo_mantenimiento	area	fecha	costo	duracion_horas	tecnico	estado	satisfacc
0	MNT-0001	Switch	Preventivo	FADCOM	2026-03-16	56.64	1.9	Ana Torres	En proceso	
1	MNT-0002	Router	Preventivo	GTSI	2026-01-28	53.86	2.3	Sofia Leon	Completado	
2	MNT-0003	Proyector	Preventivo	FIMCP	2026-03-17	83.42	1.4	Maria Cedeño	Completado	
3	MNT-0004	Desktop	Preventivo	FCNM	2026-01-31	79.54	1.7	Ana Torres	Completado	
4	MNT-0005	Switch	Correctivo	GTSI	2026-04-11	117.38	2.7	Ana Torres	Completado	

5. Uso de NumPy

NumPy se utilizó en operaciones numéricas sobre los costos. Se calcularon la media y la mediana, obteniendo una media de **84.84** y una mediana de **83.96**. También se utilizó **np.select()** para clasificar los costos en Bajo, Medio y Alto.



The screenshot shows a Google Colab notebook titled "Deber_8_P3_Mejora_Analitica.ipynb". The left sidebar displays the file explorer with a folder named "sample_data" containing a file "mantenimientos.csv". The main area shows two code cells. Cell [21] calculates the mean and median of the "costo" column using NumPy, resulting in a mean of 84.84 and a median of 83.96. Cell [22] defines conditions for classifying costs into "Bajo", "Medio", and "Alto" categories using NumPy's `np.select()` function.

```
[21] ✓ 0 s
# Convertir los costos a un arreglo de NumPy
costos = df_limpio["costo"].to_numpy()

# Calcular media y mediana
media_costos = np.mean(costos)
mediana_costos = np.median(costos)

print("Media de costos:", round(media_costos, 2))
print("Mediana de costos:", round(mediana_costos, 2))

Media de costos: 84.84
Mediana de costos: 83.96

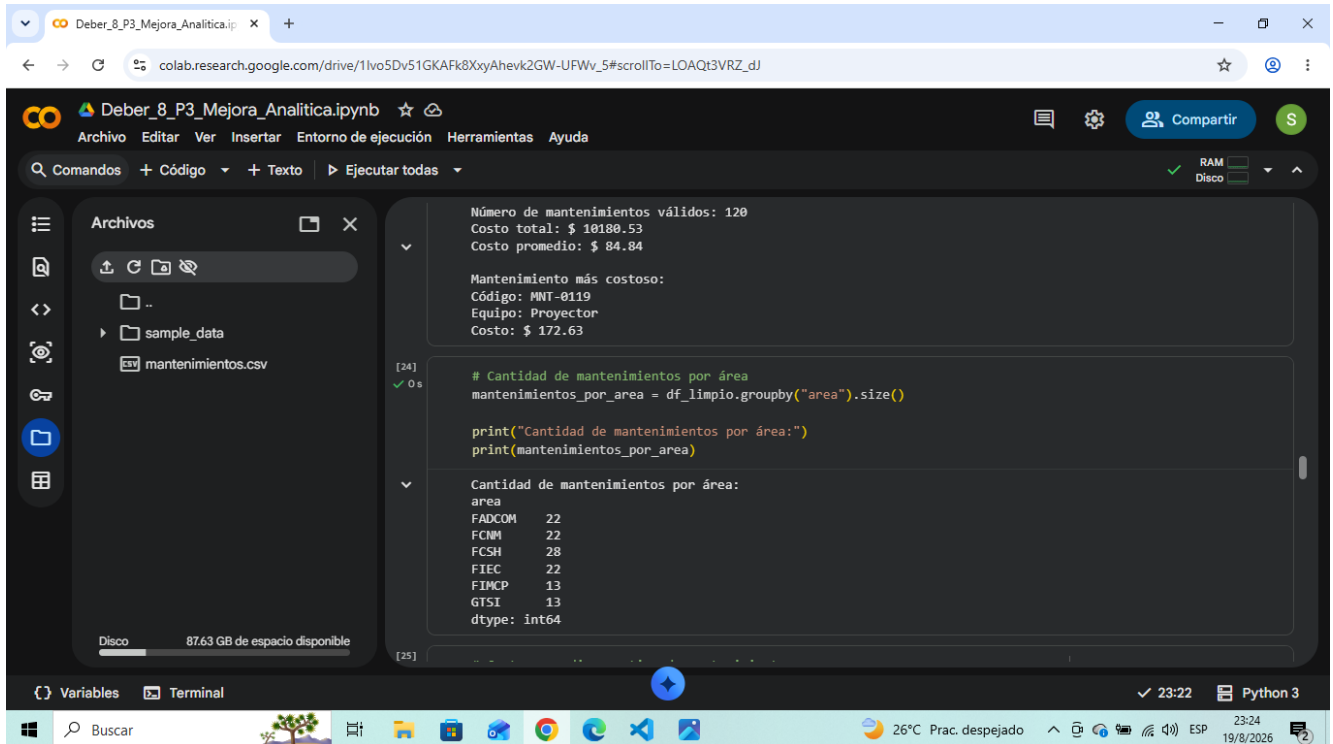
[22] ✓ 0 s
# Condiciones para clasificar los costos
condiciones = [
    df_limpio["costo"] < 60,
    (df_limpio["costo"] >= 60) & (df_limpio["costo"] <= 100),
    df_limpio["costo"] > 100
]

# Categorías
categorias = ["Bajo", "Medio", "Alto"]

# Crear una nueva columna usando NumPy
```

6. Indicadores principales con pandas

Con el DataFrame limpio se obtuvieron **120 mantenimientos validos**, un costo total de **\$10,180.53** y un costo promedio de **\$84.84**. El mantenimiento mas costoso corresponde al codigo **MNT-0119**, equipo **Proyector**, con un costo de **\$172.63**.



The screenshot shows a Google Colab notebook titled "Deber_8_P3_Mejora_Analitica.ipynb". The left sidebar displays the file explorer with a folder named "sample_data" containing a file "mantenimientos.csv". The main area shows the execution of a code cell. The output of the code is displayed in a scrollable box, showing the results of the pandas analysis.

```
[24] ✓ 0 s
```

```
# Cantidad de mantenimientos por área
mantenimientos_por_area = df_limpio.groupby("area").size()

print("Cantidad de mantenimientos por área:")
print(mantenimientos_por_area)
```

```
Cantidad de mantenimientos por área:
area
FADCOM    22
FCNM      22
FCSH      28
FIEC      22
FIMCP     13
GTSI       13
dtype: int64
```

The output also includes a summary of the data:

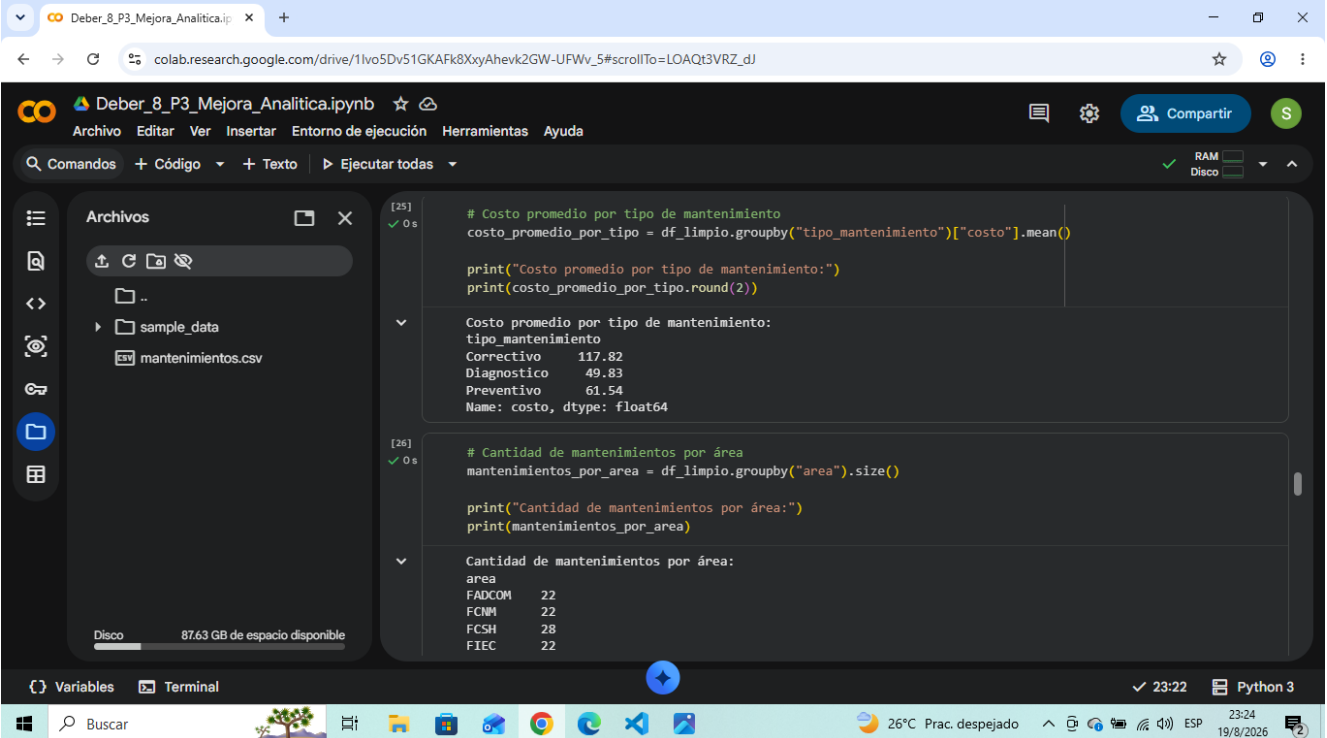
```
Número de mantenimientos válidos: 120
Costo total: $ 10180.53
Costo promedio: $ 84.84

Mantenimiento más costoso:
Código: MNT-0119
Equipo: Proyector
Costo: $ 172.63
```

The bottom of the notebook shows the status bar with "Variables", "Terminal", and "Python 3" tabs. The system tray at the bottom indicates a temperature of 26°C, a clear sky, and the date 19/8/2026.

7. Agrupaciones con pandas

Se calculo la cantidad de mantenimientos por area y el costo promedio por tipo de mantenimiento mediante **groupby()**. Los costos promedio obtenidos fueron: Correctivo \$117.82, Diagnostico \$49.83 y Preventivo \$61.54.



The screenshot shows a Google Colab notebook titled "Deber_8_P3_Mejora_Analitica.ipynb". The left sidebar shows a file named "mantenimientos.csv" in the "sample_data" folder. The main area displays two code cells and their outputs.

Cell [25]: Calculates the average cost by maintenance type.

```
# Costo promedio por tipo de mantenimiento
costo_promedio_por_tipo = df_limpio.groupby("tipo_mantenimiento")["costo"].mean()

print("Costo promedio por tipo de mantenimiento:")
print(costo_promedio_por_tipo.round(2))
```

Output:

```
Costo promedio por tipo de mantenimiento:
tipo_mantenimiento
Correctivo      117.82
Diagnostico     49.83
Preventivo      61.54
Name: costo, dtype: float64
```

Cell [26]: Calculates the number of maintenance records by area.

```
# Cantidad de mantenimientos por área
mantenimientos_por_area = df_limpio.groupby("area").size()

print("Cantidad de mantenimientos por área:")
print(mantenimientos_por_area)
```

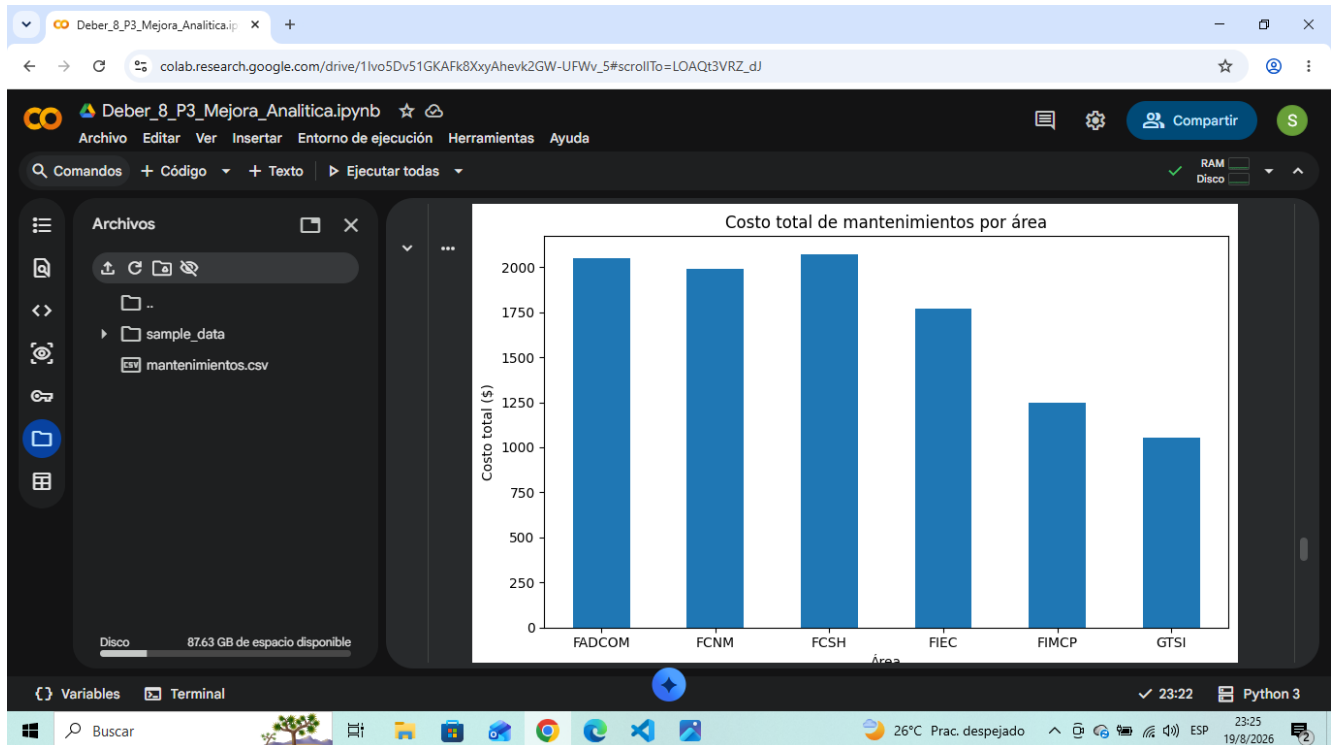
Output:

```
Cantidad de mantenimientos por área:
area
FADCOM    22
FCNM      22
FCSH      28
FIEC      22
```

The bottom of the image shows the Windows taskbar with the time 23:22 and date 19/8/2026.

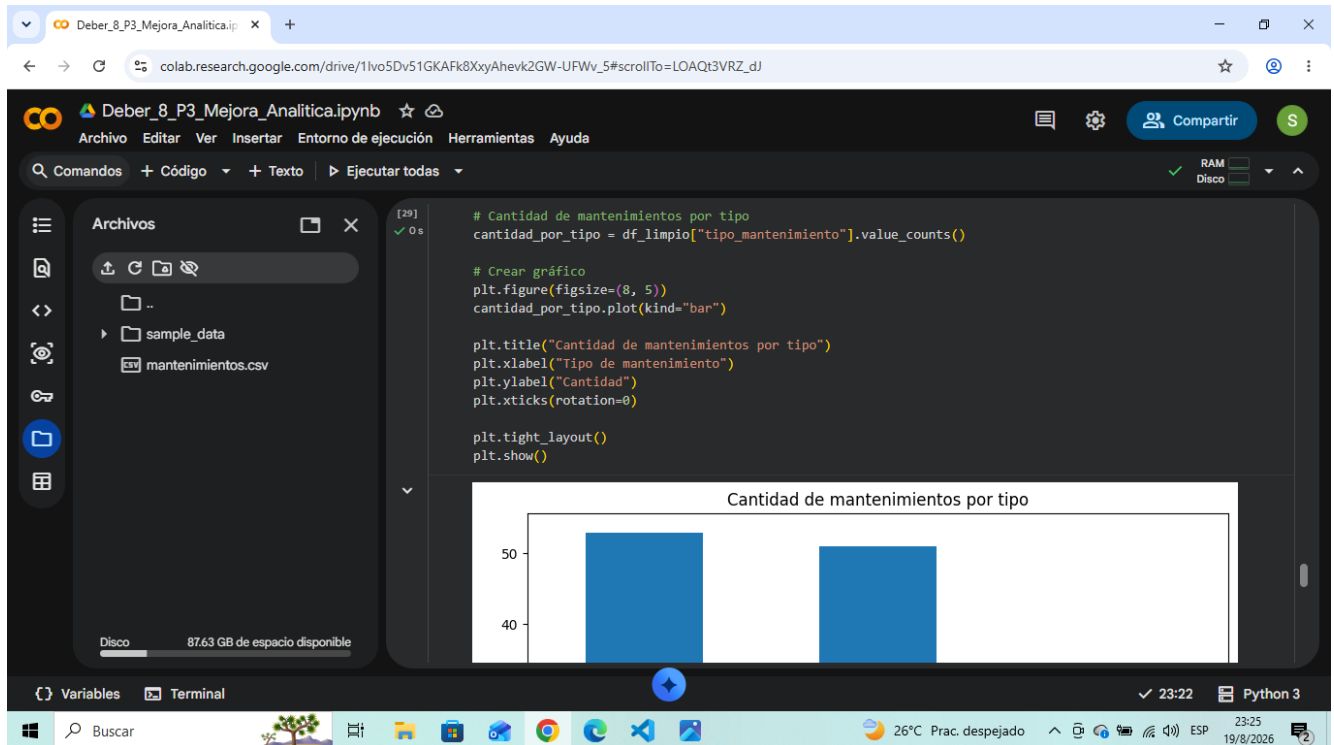
8. Grafico: costo total por area

El primer grafico de barras representa el costo total de los mantenimientos agrupado por area. Incluye titulo, etiquetas de los ejes y una presentacion legible.



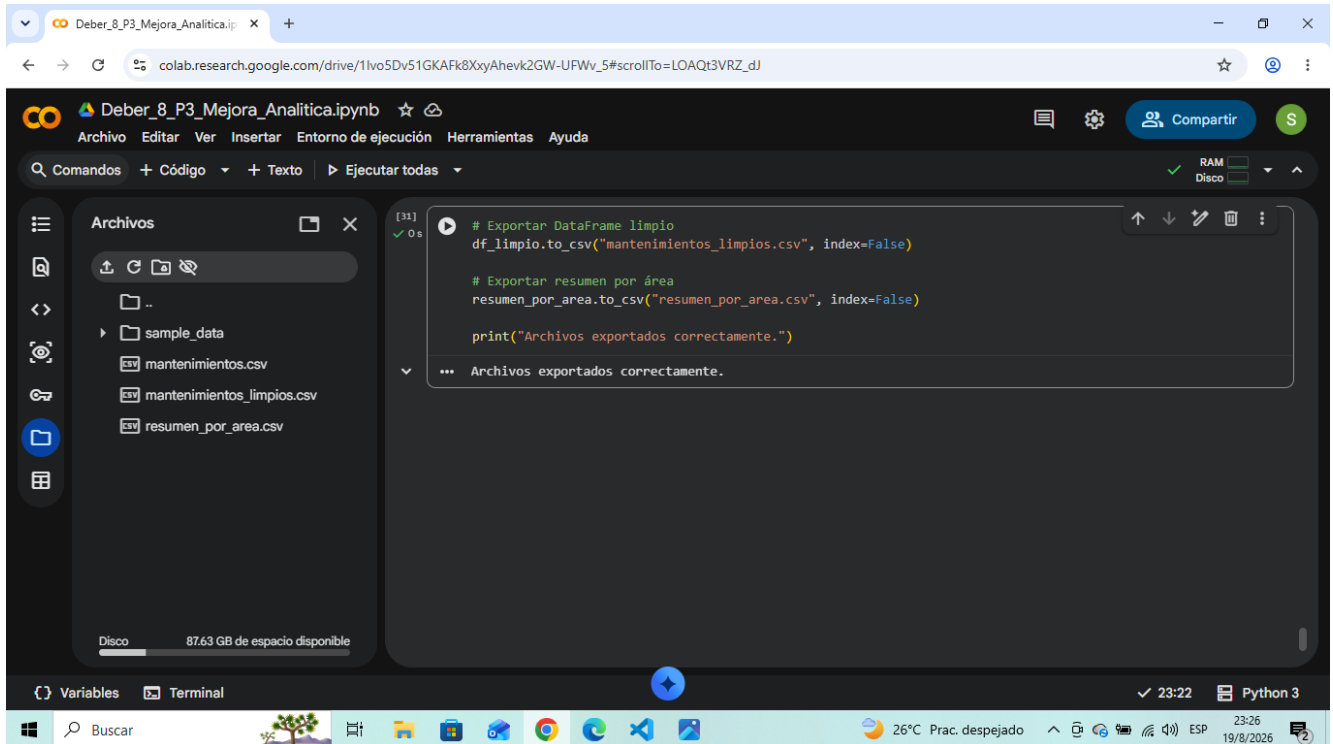
9. Grafico: cantidad por tipo

El segundo grafico representa la cantidad de mantenimientos por tipo. Este grafico permite comparar visualmente la frecuencia de los mantenimientos registrados.



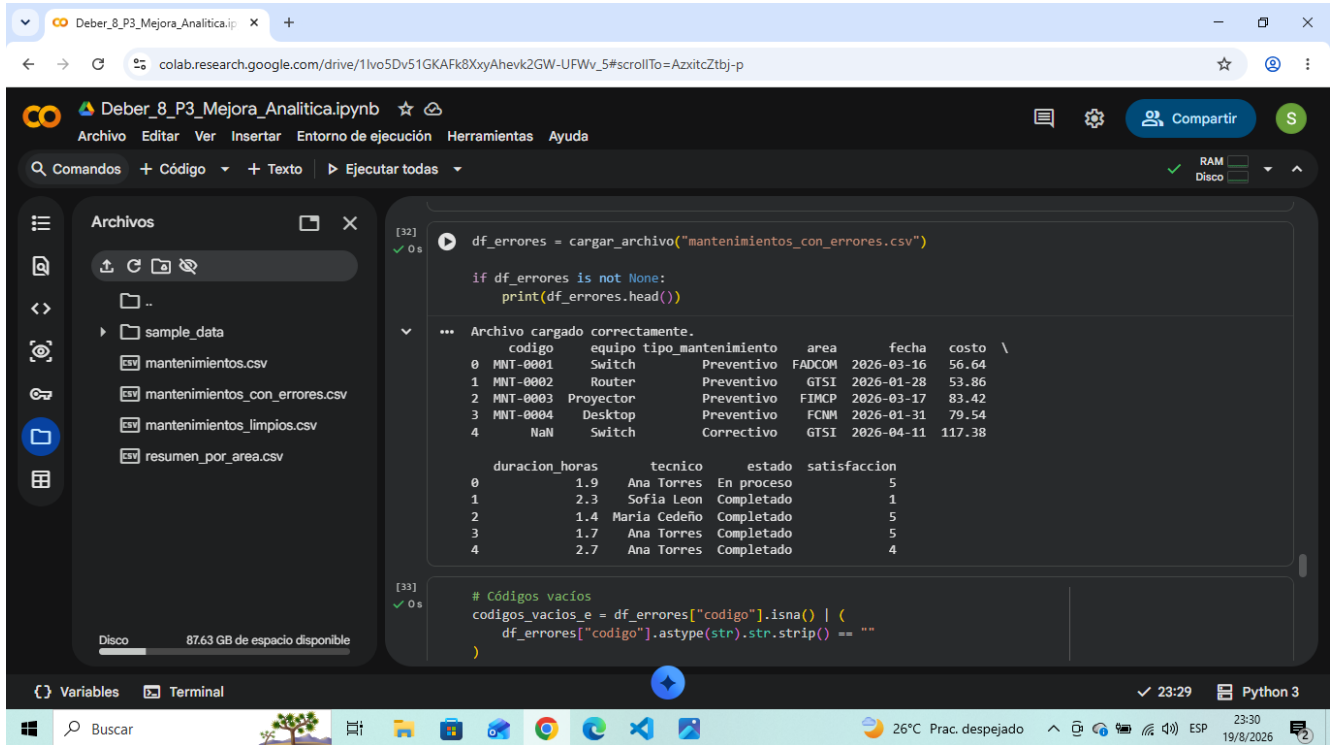
10. Exportacion de resultados

Los resultados se exportaron a los archivos **mantenimientos_limpios.csv** y **resumen_por_area.csv** mediante **to_csv()**. La captura muestra la confirmacion de la exportacion y los archivos generados en Google Colab.



11. Prueba con datos erróneos

Se cargo `mantenimientos_con_errores.csv` para comprobar las validaciones. El archivo puede leerse sin que el programa se detenga, aun cuando contiene datos incorrectos de forma deliberada.



The screenshot shows a Google Colab notebook titled "Deber_8_P3_Mejora_Analitica.ipynb". The left sidebar displays a file explorer with the following files: `mantenimientos.csv`, `mantenimientos_con_errores.csv`, `mantenimientos_limpios.csv`, and `resumen_por_area.csv`. The main code cell [32] contains the following Python code:

```
[32] df_errores = cargar_archivo("mantenimientos_con_errores.csv")
      if df_errores is not None:
          print(df_errores.head())
```

The output of the code cell shows a message "Archivo cargado correctamente." followed by a preview of the DataFrame:

	codigo	equipo	tipo_mantenimiento	area	fecha	costo
0	MNT-0001	Switch	Preventivo	FADCOM	2026-03-16	56.64
1	MNT-0002	Router	Preventivo	GTISI	2026-01-28	53.86
2	MNT-0003	Proyector	Preventivo	FIMCP	2026-03-17	83.42
3	MNT-0004	Desktop	Preventivo	FCNM	2026-01-31	79.54
4	NaN	Switch	Correctivo	GTISI	2026-04-11	117.38

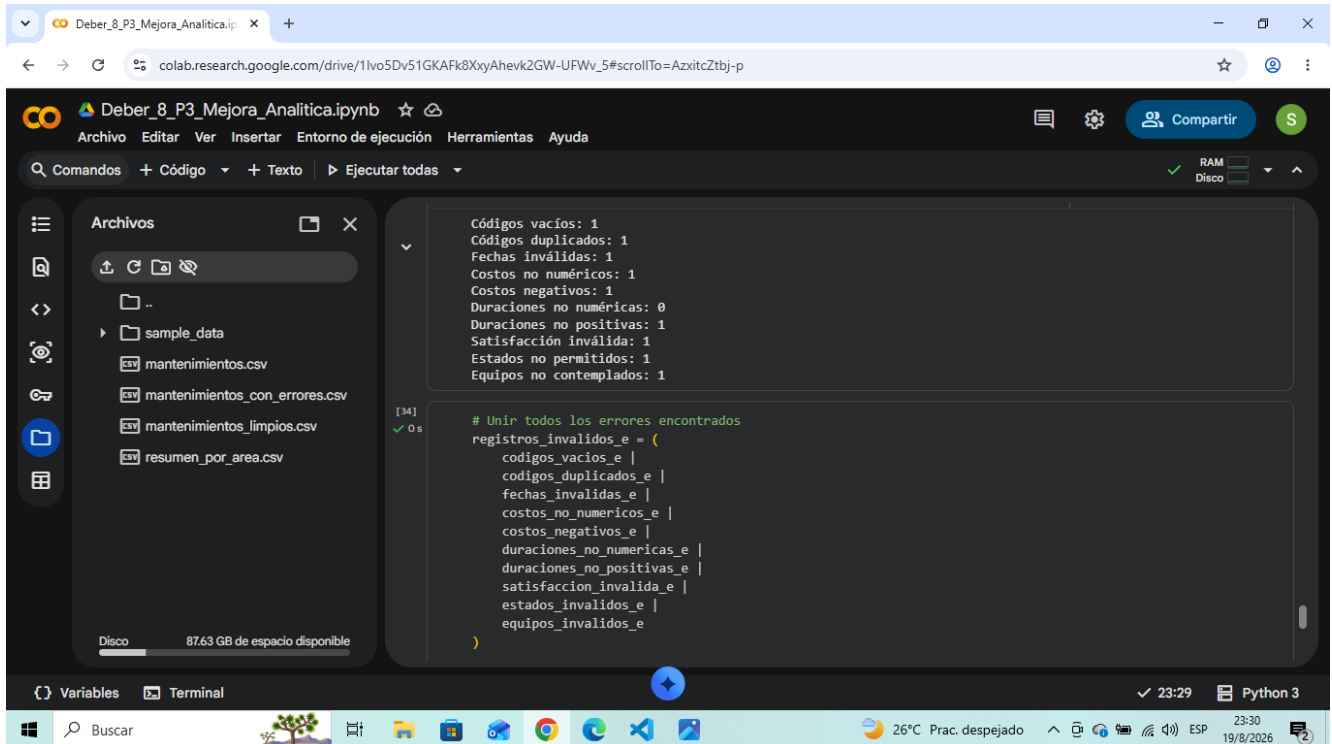
Below the DataFrame preview, the code cell [33] contains the following Python code:

```
[33] # Códigos vacíos
      codigos_vacios_e = df_errores["codigo"].isna() | (
          df_errores["codigo"].astype(str).str.strip() == ""
      )
```

The bottom of the notebook shows the status bar with "Variables", "Terminal", and "Python 3". The Windows taskbar at the bottom indicates the system time as 23:30 on 19/8/2026.

12. Errores detectados

Las validaciones detectaron código vacío, código duplicado, fecha inválida, costo no numérico, costo negativo, duración no positiva, satisfacción inválida, estado no permitido y equipo no contemplado. No se detectaron duraciones no numéricas en este conjunto.



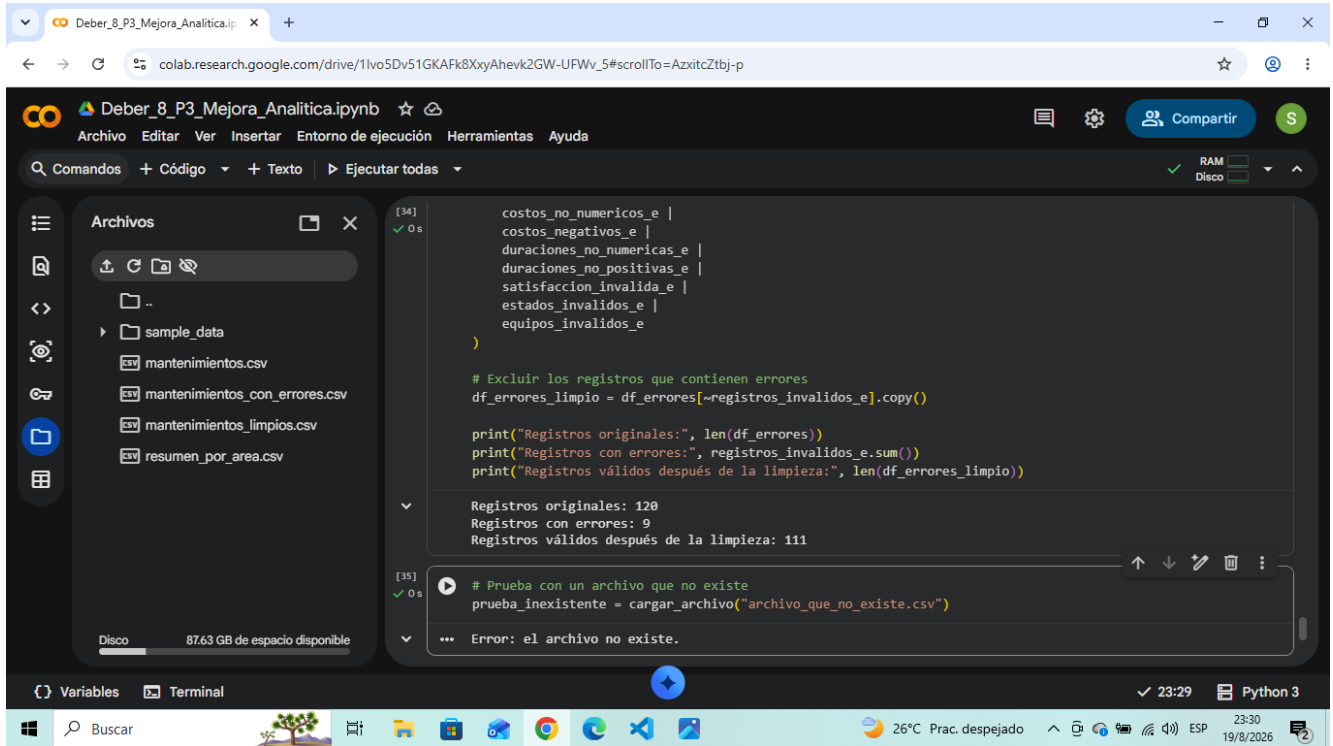
The screenshot shows a Google Colab notebook titled "Deber_8_P3_Mejora_Analitica.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" containing several CSV files: "mantenimientos.csv", "mantenimientos_con_errores.csv", "mantenimientos_limpios.csv", and "resumen_por_area.csv". The main area shows the output of a Python script. The first cell displays a list of error types and their counts: "Códigos vacíos: 1", "Códigos duplicados: 1", "Fechas inválidas: 1", "Costos no numéricos: 1", "Costos negativos: 1", "Duraciones no numéricas: 0", "Duraciones no positivas: 1", "Satisfacción inválida: 1", "Estados no permitidos: 1", and "Equipos no contemplados: 1". The second cell shows a Python dictionary comprehension that combines these error types into a single list: "registros_invalidos_e = (codigos_vacios_e | codigos_duplicados_e | fechas_invalidas_e | costos_no_numericos_e | costos_negativos_e | duraciones_no_numericas_e | duraciones_no_positivas_e | satisfaccion_invalida_e | estados_invalidos_e | equipos_invalidos_e)". The bottom status bar indicates the system is running Python 3, with a temperature of 26°C and a date of 19/8/2026.

```
Códigos vacíos: 1
Códigos duplicados: 1
Fechas inválidas: 1
Costos no numéricos: 1
Costos negativos: 1
Duraciones no numéricas: 0
Duraciones no positivas: 1
Satisfacción inválida: 1
Estados no permitidos: 1
Equipos no contemplados: 1

# Unir todos los errores encontrados
registros_invalidos_e = (
    codigos_vacios_e |
    codigos_duplicados_e |
    fechas_invalidas_e |
    costos_no_numericos_e |
    costos_negativos_e |
    duraciones_no_numericas_e |
    duraciones_no_positivas_e |
    satisfaccion_invalida_e |
    estados_invalidos_e |
    equipos_invalidos_e
)
```

13. Limpieza y archivo inexistente

Al combinar las reglas se identificaron **9 registros con errores** de un total de 120, quedando **111 registros validos** despues de la limpieza. Tambien se realizo la prueba obligatoria con un archivo inexistente, que fue controlada correctamente mediante **FileNotFoundError** sin detener abruptamente el programa.



The screenshot shows a Google Colab notebook titled "Deber_8_P3_Mejora_Analitica.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" containing four CSV files: "mantenimientos.csv", "mantenimientos_con_errores.csv", "mantenimientos_limpios.csv", and "resumen_por_area.csv". The main editor area contains two code cells. Cell [34] defines a list of error patterns, filters a DataFrame to remove records containing these errors, and prints the counts: "Registros originales: 120", "Registros con errores: 9", and "Registros válidos después de la limpieza: 111". Cell [35] attempts to load a non-existent file "archivo_que_no_existe.csv", which results in a "FileNotFoundError" message: "Error: el archivo no existe." The bottom status bar indicates the environment is Python 3 and the time is 23:29.

```
[34]
costos_no_numericos_e |
costos_negativos_e |
duraciones_no_numericas_e |
duraciones_no_positivas_e |
satisfaccion_invalida_e |
estados_invalidos_e |
equipos_invalidos_e
)

# Excluir los registros que contienen errores
df_errores_limpio = df_errores[~registros_invalidos_e].copy()

print("Registros originales:", len(df_errores))
print("Registros con errores:", registros_invalidos_e.sum())
print("Registros válidos después de la limpieza:", len(df_errores_limpio))

Registros originales: 120
Registros con errores: 9
Registros válidos después de la limpieza: 111

[35]
# Prueba con un archivo que no existe
prueba_inexistente = cargar_archivo("archivo_que_no_existe.csv")

... Error: el archivo no existe.
```

14. Ciclo y estructuras de control

El programa utiliza un ciclo **for** para recorrer la cantidad de mantenimientos por area. Dentro del ciclo se emplea una estructura **if/elif/else** para clasificar cada area segun su cantidad de mantenimientos. De esta forma, el ciclo tiene una finalidad real dentro del analisis y cumple el requisito de estructuras de control.

Conclusion

La solucion permite cargar los registros, verificar su calidad, detectar y excluir datos invalidos, calcular indicadores con pandas y NumPy, visualizar resultados con Matplotlib y exportar los datos procesados. Las pruebas realizadas demuestran que el notebook funciona tanto con el archivo valido como con el archivo que contiene errores y que maneja correctamente un archivo inexistente.